

FAST NUCES

Karachi Campus



Railway Signal Coordination System

PROJECT REPORT

CS-2006: Operating Systems

GROUP MEMBERS	
Maryam Saleem	24K-0779
Umaina Manzoor	24K-0501
Aiman Misbah	24K-0842

Submission Date: 11th May 2026

1. Executive Summary

Overview

This project presents a multi-threaded Railway Signal Coordination System implemented in C using Operating System concepts such as thread synchronization, mutual exclusion, semaphores, and deadlock prevention. Each train is modeled as an independent thread, while railway tracks act as shared resources. A graphical interface built using Raylib visualizes real-time train movement, synchronization behavior, and resource allocation.

The system demonstrates how operating system principles are applied to manage concurrent execution units(threads), accessing shared resources safely, efficiently, and without deadlocks.

Key Features

- Multi-threaded train simulation using POSIX threads
- Semaphore-based signal control system
- Mutex-protected track segments
- FIFO-based scheduling for fairness
- Deadlock prevention through ordered locking
- Real-time graphical visualization using Raylib
- Live system statistics (waiting time, track usage, junction status)

2. Introduction

Background

Operating Systems manage multiple processes accessing shared resources concurrently. Improper synchronization can lead to race conditions, deadlocks, and starvation. This project simulates these problems in a railway system where trains must safely traverse shared tracks.

Objective

- Simulate concurrent train execution using threads
- Implement synchronization using mutexes and semaphores
- Prevent deadlocks using ordered resource acquisition
- Visualize OS scheduling and synchronization behavior

3. Project Description & Scope

Scope

- Single railway network simulation

- Multi-train concurrent execution
- Safe resource sharing using OS mechanisms
- Real-time visualization of system state

Exclusions

- No distributed system support
- No external network communication
- No AI-based routing

Tools & Technologies

Technology	Purpose
C Programming Language	Core implementation language
POSIX Threads (pthreads)	Multi-threading and concurrency
Mutex Locks & Semaphores	Synchronization primitives
Raylib Graphics Library	Real-time GUI visualization
GCC Compiler	Compilation and linking
Makefile Build System	Automated build management

4. Methodology

System Architecture

The system is divided into three layers:

Layer	Name	Components
1	Process Layer	Train Threads
2	Synchronization Layer	Mutex + Semaphore System
3	Visualization Layer	Raylib GUI

Execution Flow

1. Train threads are created in main.c
2. Each train selects a destination track
3. Semaphore is requested before movement
4. Mutex locks control track access
5. GUI visualizes system state in real time

- Resources are released after movement

Scheduling

- FIFO-based resource allocation ordering
- Prevents starvation and ensures fairness

5. Project Implementation

5.1 Core Concepts Implemented

1. Thread Creation (Process Model) — main.c

```
pthread_create(&threads[i], NULL, train, &trains[i]);  
pthread_join(threads[i], NULL);
```

Purpose: Each train is implemented as an independent OS process-like thread, simulating true concurrency. This models how an operating system executes multiple processes simultaneously while sharing resources.

Implementation: Each train structure is passed to `pthread_create`, creating an independent execution flow. Every thread runs the `train()` function which controls lifecycle states such as signal waiting, movement, and track acquisition. The main thread waits using `pthread_join` to ensure all trains complete execution before program termination.

2. Train State Machine — train.h

```
typedef enum {  
    WAITING_SIGNAL,  
    WAITING_TRACK,  
    MOVING,  
    ARRIVING,  
    FINISHED  
} TrainState;
```

Purpose: Defines the complete lifecycle of a train as a process state model in an operating system.

Implementation: Each train transitions through:

- `WAITING_SIGNAL` → waiting for scheduling permission
- `WAITING_TRACK` → waiting for resource availability
- `MOVING` → executing critical section (track traversal)
- `ARRIVING` → completing execution cycle
- `FINISHED` → termination state

This mimics OS process state transitions (Ready → Running → Blocked → Terminated).

3. Semaphore-Based Signal System — signal.c

```
sem_init(&signalSemaphore, 0, 4);  
sem_wait(&signalSemaphore);  
sem_post(&signalSemaphore);
```

Purpose: Controls how many trains can enter critical railway sections simultaneously.

Implementation: A counting semaphore limits concurrent access to 4 trains. Each train must acquire permission before entering critical track zones, ensuring controlled concurrency and preventing congestion.

4. Mutex-Based Track Locking — track.c / train.c

```
pthread_mutex_lock(&tracks[i].mutex);  
pthread_mutex_unlock(&tracks[i].mutex);
```

Purpose: Ensures mutual exclusion on track segments.

Implementation: Each track segment is a shared resource protected by a mutex. Only one train can occupy a track segment at a time, preventing race conditions and inconsistent state updates.

5. Deadlock Prevention (Ordered Locking) — train.c

```
int low = (start < end) ? start : end;  
int high = (start > end) ? start : end;  
  
for(int i = low - 1; i <= high - 1; i++)  
    pthread_mutex_lock(&tracks[i].mutex);
```

Purpose: Eliminates circular wait condition in multi-resource acquisition.

Implementation: All trains lock track segments in ascending order. This global ordering removes deadlock possibility entirely by breaking circular dependency.

6. FIFO Scheduling System — train.c

```
if(global_trains[i].arrival_time < oldest_time)
```

Purpose: Ensures fair scheduling among trains.

Implementation: Each train is assigned an incremental `arrival_time` using a global counter protected by `queue_mutex`. The scheduler repeatedly scans the `global_trains[]` array to find the train with the smallest arrival timestamp among waiting trains.

Instead of a strict queue-based FCFS structure, the system uses a **dynamic selection approach with continuous evaluation and retry (usleep-based backoff)**. This ensures fairness while allowing synchronization with real-time state changes of trains.

7. Shared Memory Communication — train.c / gui.c

```
extern Train* global_trains;  
extern int track_status[TOTAL_TRACKS];
```

Purpose: Enables real-time synchronization between backend logic and GUI rendering.

Implementation: Shared structures like:

- `global_trains`
- `track_status`
- `visual_trains`

allow GUI and train threads to operate on the same memory space, simulating shared memory multiprocessing.

8. Critical Section Protection — train.c

```
pthread_mutex_lock(&track_status_mutex);  
track_status[k] = t->id;  
pthread_mutex_unlock(&track_status_mutex);
```

Purpose: Ensures safe and synchronized modification of shared system state variables accessed concurrently by multiple train threads.

Implementation: A dedicated mutex (`track_status_mutex`) protects updates to the shared `track_status` array. Before modifying shared track occupancy information, a train thread acquires the mutex lock to guarantee exclusive access. After updates are completed, the mutex is released. This prevents race conditions, inconsistent resource states, and simultaneous conflicting writes from concurrent threads.

9. GUI Thread Execution — gui.c

```
pthread_create(&gui_thread, NULL, startGUI, NULL);
```

Purpose: Separates visualization logic from backend simulation logic to enable independent real-time system monitoring and rendering.

Implementation: The GUI thread is created in `main.c` using `pthread_create`, while the rendering and visualization logic is implemented inside `gui.c`. The GUI continuously updates the graphical state of trains, tracks, junctions, and system statistics using Raylib. It operates concurrently with backend train threads and synchronizes using shared global memory structures..

10. Junction as Critical Section — gui.c

```
junctions[target_junction].occupied_by = actual_train->id;  
junctions[visual_trains[i].target_junction].occupied_by = -1;
```

Purpose: Represents railway junctions as shared crossing points where only one train should visually occupy the intersection at a time.

Implementation: The junction control is handled primarily in the **GUI layer**, not the backend train logic. When a train enters a junction region, the GUI marks it as occupied using `occupied_by = train_id`. Once the train completes traversal, the junction is reset to `-1`.

This ensures **visual mutual exclusion at intersections**, preventing overlapping train rendering at junctions. However, this is a **GUI-level coordination mechanism**, not a kernel-level synchronization primitive like mutex or semaphore.

11. Track Resource Initialization — track.c

```
pthread_mutex_init(&tracks[i].mutex, NULL);
```

Purpose: Initializes synchronization primitives for all shared track resources.

Implementation: At system startup, each track is assigned a mutex lock, preparing it for safe concurrent access during simulation execution.

12. Logging System —train.c / main.c

```
fprintf(log_file, "[%s] Train %d: %s\n", buf, t->id, msg);
```

Purpose: Maintains a persistent execution log of important system events for debugging, monitoring, and post-execution analysis.

Implementation: The logging system is initialized and closed in `main.c`, while runtime train activity logging is handled in `train.c` through the `log_train()` function. Major train events such as departures and arrivals are written to `railway_log.txt` along with timestamps. Mutex protection ensures synchronized console and file output during concurrent execution. .

13. Makefile Build System — Makefile

```
$(CC) $(CFLAGS) $(SRC) -o $(TARGET) $(LIBS)
```

Purpose: Automates compilation and linking of the entire project.

Implementation: All source files are compiled together and required libraries (pthread, raylib, math, etc.) are linked to produce the final executable.

6. Important Parts of the Code

Component	File	Purpose
Train Logic System	train.c	Handles train lifecycle, movement logic, synchronization, and scheduling
Signal Control System	signal.c	Controls train entry into critical regions using semaphores
Track Resource System	track.c	Initializes mutex-protected shared track resources
GUI Visualization Engine	gui.c	Provides real-time visualization and system monitoring
System Initialization	main.c	Initializes system components, creates threads, and manages execution lifecycle
Scheduling System	train.c	Implements FIFO-based resource allocation ordering
Shared State System	train.c/gui.c	Defines shared train structures and global shared memory access
Deadlock-Free Locking	train.c	Implements ordered mutex acquisition for deadlock prevention
Logging System	main.c	System trace recording
Junction System	gui.c	Intersection critical section handling

7. Roles and Responsibilities

Member	Responsibilities
Umair Manzoor	GUI system (gui.c, gui.h) — Real-time visualization and rendering
Aiman Misbah	Train logic system (train.c, train.h) — Thread execution and scheduling logic
Maryam Saleem	Synchronization system (signal.c, track.c) — System initialization and logging (main.c)

8. Results

- System successfully simulates **true multithreaded railway execution**
- Deadlock is completely prevented using **ordered locking strategy**
- FIFO scheduling ensures **fair execution using arrival counters**
- GUI correctly reflects **real-time shared memory state**

- Junction system ensures **safe intersection traversal**
- Semaphore + mutex hybrid system ensures **high concurrency safety**

9. Challenges Faced

- Managing **multiple synchronization layers** (mutex + semaphore + GUI lock)
- Ensuring consistency between **backend threads** and **GUI rendering**
- Preventing race conditions in **shared memory structures**
- Designing **deadlock-free multi-resource locking strategy**
- Maintaining smooth **real-time rendering at 60 FPS with thread updates**
- Handling **junction collision avoidance** in **dynamic movement system**

10. Conclusion

This project successfully demonstrates the application of core operating system concepts including multithreading, synchronization, mutual exclusion, scheduling, and deadlock prevention. The system effectively simulates a real-world railway coordination environment with accurate process and resource management behavior.

Through the implementation of POSIX threads, semaphores, mutex locks, and FIFO scheduling, the project provides a working model of how modern operating systems handle concurrent resource access safely and efficiently. The Raylib-based graphical interface further enhances the educational value by making abstract OS concepts visually tangible.

11. Outputs





